PROJET 03 / ADMINISTRATION LINUX

LINUX
LOAD BALANCING

Apache2 et HAProxy sur Ubuntu Server

Utilisateurs / Groupes / Permissions / HTTP

RÉALISÉ PAR

Rebecca BOSAKE

Formation : BTS SIO — année à compléter

Établissement : à compléter

Année scolaire : à compléter

Date de remise : à compléter

COMPTE RENDU TECHNIQUE ET GUIDE DE REPRISE

Trois serveurs Ubuntu / Réseau du TP : 192.168.179.0/24

25 captures originales intégrées, commentées et légendées.

Un load balancer unique : aucune redondance Heartbeat déployée.

SOMMAIRE

Organisation du livrable

Partie	Pages
Contexte, objectifs et architecture	3–4
Préparation réseau et connectivité	5–7
Utilisateurs, groupes et administration	8–12
Permissions, setgid et travail collaboratif	13–15
Apache2, publication et tests directs	16–20
HAProxy, configuration et équilibrage	21–25
Pare-feu UFW et sécurisation	26–27
Recette, diagnostic et corrections	28–29
Questions de synthèse et conclusion	30–31
Index des 25 captures originales	32

Lecture du document
Les images viennent du **TP_Linux_loadbalancer_REBECCA.pdf**, 10 pages. La **procédure de reprise** harmonise les commandes et les rôles. Les résultats attendus ne sont pas des mesures obtenues sur vos machines pendant la génération.

Ne pas confondre modèle et réalisation
Le modèle « Livrable HAProxy.pdf » sert de référence de présentation. Son cluster Debian à deux proxys avec Heartbeat ne fait pas partie de ce TP Ubuntu à trois serveurs. Les captures d’un autre groupe ne sont pas utilisées comme preuves du travail de Rebecca.

Les légendes renvoient aux pages du TP original. Les 25 images sont conservées sans retouche des valeurs ; les corrections figurent dans les explications.

01 — PRÉSENTATION

Contexte et objectifs

L'atelier porte sur l'administration de trois serveurs Ubuntu partageant un réseau de laboratoire. Deux machines hébergent des pages web avec Apache2. La troisième reçoit les requêtes HTTP et les répartit à l'aide de HAProxy.

Une première partie traite des comptes locaux, des groupes et des permissions Linux. La seconde relie le service web à l'équilibrage de charge et à la tolérance à la panne d'un backend.

Objectifs techniques

- Créer **alice**, **bob** et **adminweb** sur les trois serveurs.
- Organiser les développeurs dans **web** et les administrateurs dans **admins**.
- Créer **/srv/web** et expliquer les permissions et le bit setgid.
- Installer Apache2 sur les deux machines web et différencier leur contenu.
- Configurer un frontend HTTP et un backend HAProxy de deux serveurs.
- Vérifier les réponses HTTP, la répartition et le retrait d'un serveur indisponible.
- Configurer UFW sans perdre les accès d'administration.

Méthode de travail

Chaque étape présente l'objectif, les commandes, les captures disponibles et les vérifications. La reprise doit se faire sur des machines de laboratoire avec des sauvegardes avant modification.

> **Disponibilité limitée par lb**
> HAProxy peut continuer à servir le site si un backend tombe et si l'autre reste accessible. En revanche, **la panne de lb bloque l'accès via lb** : aucun second proxy ni mécanisme de reprise d'IP n'est prévu.

01 — ARCHITECTURE

Plan réseau et rôle des machines

Machine du TP	Rôle retenu	IPv4
lb / test	HAProxy	192.168.179.143
web1 / alltp	Apache2 — WEB 1	192.168.179.142
web2 / serv02	Apache2 — WEB 2	192.168.179.136

Le plan conserve les adresses annoncées dans le TP. Le masque **255.255.255.0 (/24)** apparaît dans les captures réseau. Le client peut être l'hôte sur un réseau host-only partagé ou une VM cliente sur le même segment.

Chemin d'une requête HTTP

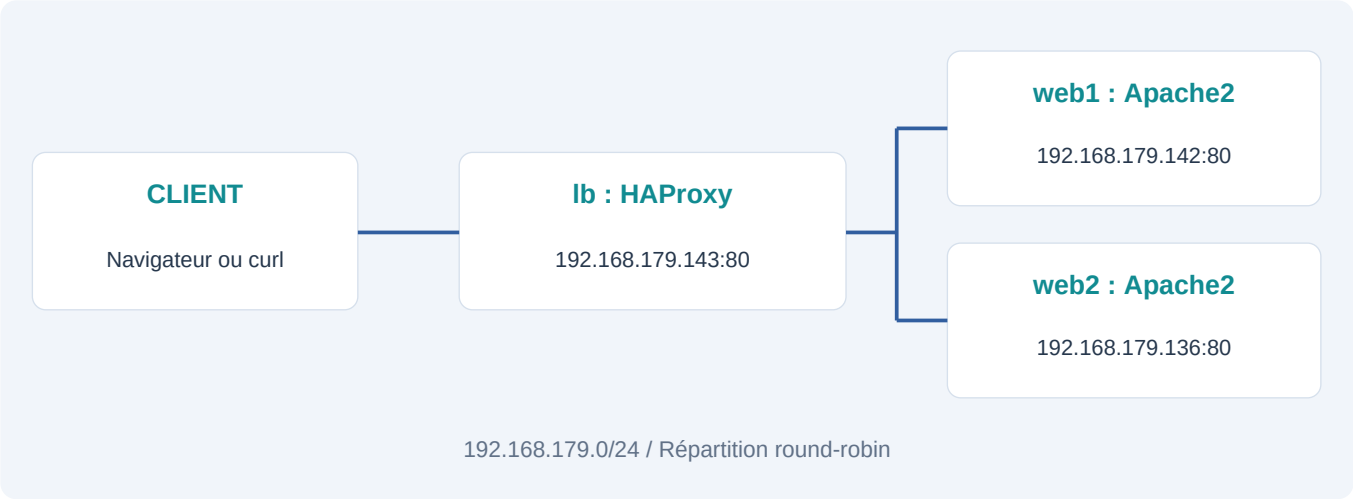


Figure A — Schéma créé pour le livrable, pas une capture de test.

Repères à harmoniser
Les pages historiques associent parfois WEB 1 à .136 et WEB 2 à .142. La reprise retient **web1 = .142** et **web2 = .136**, conformément à l'énoncé. Tester HAProxy sur **.143**, pas sur une adresse de backend.

02 — PRÉPARATION

Installation et configuration du réseau

Objectif : préparer trois Ubuntu Server avec un compte sudo. L'installation et le réseau virtuel complètent le TP, qui suppose ces prérequis disponibles.

1. Créer les VM lb, web1 et web2. Définir leur nom dans Ubuntu également : l'onglet de l'hyperviseur ne prouve pas le nom du système invité.

2. Prévoir, par exemple, 2 vCPU, 2 à 4 Go de RAM et 25 Go de disque par serveur. Adapter à l'Ubuntu installé et aux ressources de l'hôte.

3. Installer les paquets avec un accès NAT fonctionnel, puis relier les VM au même réseau privé. Le sous-réseau .179 peut être conservé si le réseau virtuel le permet.

4. Fixer les IP .143, .142 et .136 hors conflit avec le DHCP. Si DHCP est conservé, prévoir des réservations plutôt que supposer les adresses permanentes.

Exemple Netplan isolé : lb

Relever le nom de l'interface avec **ip -br link**. Modifier le fichier existant sans définition contradictoire. L'interface ens33 ci-dessous est à adapter.

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.179.143/24]

sudo netplan generate
sudo netplan try
ip -br address
```

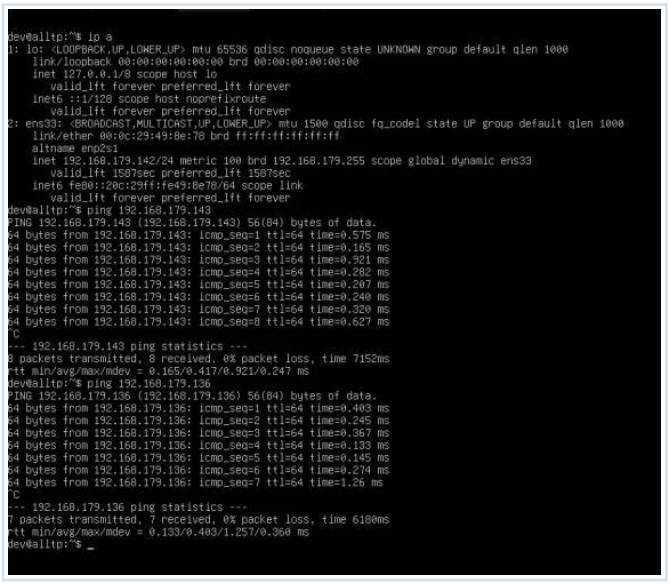
Sur les serveurs web, utiliser leurs IP respectives. Cet exemple privé ne définit pas de route Internet ni de DNS public. Préparer l'accès aux dépôts avant isolation.

> **Avant application**
> Utiliser la console de la VM pour éviter de perdre une session distante. Ne pas recopier une passerelle inconnue et vérifier le réseau après validation.

02 — RÉSEAU

Contrôles historiques sur les serveurs

Objectif : vérifier les adresses et la connectivité entre les machines avant de contrôler les services.



```
dev@alltp:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:45:8e:78 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.179.142/24 metric 100 brd 192.168.179.255 scope global dynamic ens33
       valid_lft 1587sec preferred_lft 1587sec
    inet6 fe80::20c:29ff:fe45:8e78/64 scope link
       valid_lft forever preferred_lft forever
dev@alltp:~$ ping 192.168.179.143
PING 192.168.179.143 (192.168.179.143) 56(84) bytes of data.
64 bytes from 192.168.179.143: icmp_seq=1 ttl=64 time=0.575 ms
64 bytes from 192.168.179.143: icmp_seq=2 ttl=64 time=0.165 ms
64 bytes from 192.168.179.143: icmp_seq=3 ttl=64 time=0.921 ms
64 bytes from 192.168.179.143: icmp_seq=4 ttl=64 time=0.282 ms
64 bytes from 192.168.179.143: icmp_seq=5 ttl=64 time=0.207 ms
64 bytes from 192.168.179.143: icmp_seq=6 ttl=64 time=0.240 ms
64 bytes from 192.168.179.143: icmp_seq=7 ttl=64 time=0.320 ms
64 bytes from 192.168.179.143: icmp_seq=8 ttl=64 time=0.627 ms
^C
--- 192.168.179.143 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7152ms
rtt min/avg/max/mdev = 0.165/0.417/0.921/0.247 ms
dev@alltp:~$ ping 192.168.179.136
PING 192.168.179.136 (192.168.179.136) 56(84) bytes of data.
64 bytes from 192.168.179.136: icmp_seq=1 ttl=64 time=0.403 ms
64 bytes from 192.168.179.136: icmp_seq=2 ttl=64 time=0.245 ms
64 bytes from 192.168.179.136: icmp_seq=3 ttl=64 time=0.367 ms
64 bytes from 192.168.179.136: icmp_seq=4 ttl=64 time=0.133 ms
64 bytes from 192.168.179.136: icmp_seq=5 ttl=64 time=0.145 ms
64 bytes from 192.168.179.136: icmp_seq=6 ttl=64 time=0.274 ms
64 bytes from 192.168.179.136: icmp_seq=7 ttl=64 time=1.26 ms
^C
--- 192.168.179.136 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6180ms
rtt min/avg/max/mdev = 0.133/0.403/1.257/0.360 ms
dev@alltp:~$ _
```

Capture 01 — Réseau depuis alltp.
TP original, p. 1. IP .142 ; tests vers les autres machines.



```
From 192.168.179.143 icmp_seq=11 Destination Host Unreachable
From 192.168.179.143 icmp_seq=12 Destination Host Unreachable
^C
--- 192.168.179.144 ping statistics ---
13 packets transmitted, 0 received, +12 errors, 100% packet loss, time 12280ms
pipe 4
dev@test:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:1c:9c:83 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.179.143/24 metric 100 brd 192.168.179.255 scope global dynamic ens33
       valid_lft 1517sec preferred_lft 1517sec
    inet6 fe80::20c:29ff:fe1c:9c83/64 scope link
       valid_lft forever preferred_lft forever
dev@test:~$ ping 192.168.179.136
PING 192.168.179.136 (192.168.179.136) 56(84) bytes of data.
64 bytes from 192.168.179.136: icmp_seq=1 ttl=64 time=0.668 ms
64 bytes from 192.168.179.136: icmp_seq=2 ttl=64 time=0.301 ms
64 bytes from 192.168.179.136: icmp_seq=3 ttl=64 time=0.285 ms
64 bytes from 192.168.179.136: icmp_seq=4 ttl=64 time=0.730 ms

--- 192.168.179.136 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3079ms
rtt min/avg/max/mdev = 0.285/0.496/0.730/0.204 ms
^Cdev@test:~$ ping 192.168.179.142
PING 192.168.179.142 (192.168.179.142) 56(84) bytes of data.
64 bytes from 192.168.179.142: icmp_seq=1 ttl=64 time=0.487 ms
64 bytes from 192.168.179.142: icmp_seq=2 ttl=64 time=0.919 ms
64 bytes from 192.168.179.142: icmp_seq=3 ttl=64 time=0.667 ms
^C
--- 192.168.179.142 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2038ms
rtt min/avg/max/mdev = 0.487/0.691/0.919/0.177 ms
dev@test:~$
```

Capture 02 — Réseau depuis test.
TP original, p. 1. IP .143 ; un ancien essai vers .144 échoue, puis .136 et .142 répondent.

Commandes de reprise

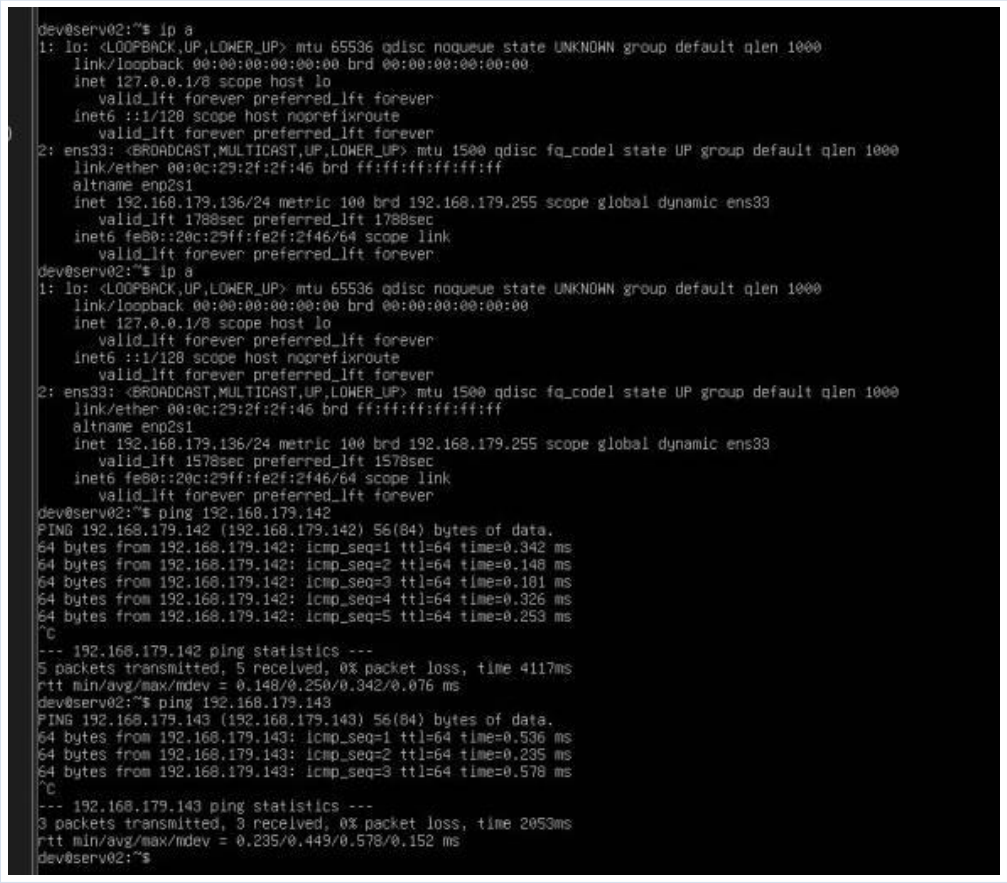
```
ip -br address
ip route
# Depuis lb :
ping -c 4 192.168.179.142
ping -c 4 192.168.179.136
```

Résultat attendu : réponses des deux serveurs, sans message d’hôte inaccessible. Si ICMP est filtré, contrôler le réseau puis TCP 80 après installation d’Apache.

> **Portée des images**
> Les réponses concernent l’ancien laboratoire. Elles ne prouvent ni la permanence du DHCP, ni le fonctionnement de la future configuration HAProxy.

02 — RÉSEAU

Connectivité de web2 et interprétation



Capture 03 — Tests réseau depuis serv02. TP original, p. 1. L'interface possède .136 et les tests vers .142 et .143 répondent.

Les niveaux de vérification

Niveau	Ce qu'il vérifie
Adresse IP	Interface active, IP attendue et masque correct.
Ping	Réponse ICMP de la destination si autorisée.
HTTP direct	Réponse d'Apache sur chaque backend.
HTTP via lb	Requête traversant HAProxy et un backend utilisable.

Un ping réussi ne prouve pas que HTTP répond ; un HTTP direct ne prouve pas la configuration du proxy. La recette sépare ces contrôles pour localiser les pannes.

03 — IDENTITÉS

Création des utilisateurs locaux

Objectif : créer les comptes demandés sur **les trois machines**. Ils sont locaux : aucune synchronisation automatique ni annuaire central n'est configuré.

Compte	Rôle	Privilèges retenus
alice	Développement	Ordinaire ; membre de web
bob	Développement	Ordinaire ; membre de web
adminweb	Administration	Membre de admins ; sudo si autorisé

```
sudo adduser alice
sudo adduser bob
sudo adduser adminweb
getent passwd alice bob adminweb
```

1. Utiliser le compte d'installation autorisé à lancer sudo. Saisir une commande distincte pour chaque compte.

2. Choisir des mots de passe robustes, sans les afficher. Vérifier les répertoires personnels créés par adduser.

3. Si un compte existe déjà, examiner ses propriétés plutôt que relancer sa création.

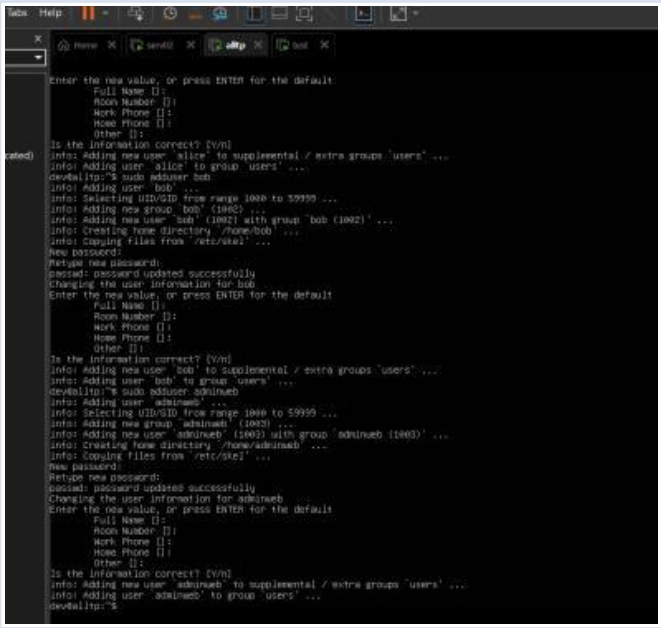
4. Répéter sur lb, web1 et web2, puis relever les comptes présents sur chacune.

> **Point à corriger : commandes mélangées**
> La mise en forme du TP rapproche parfois sudo de la commande suivante. **sudo adduser alice** crée le compte ; l'ajout à un groupe se fait séparément. Alice et Bob ne doivent pas recevoir sudo par copie d'une ligne ambiguë.

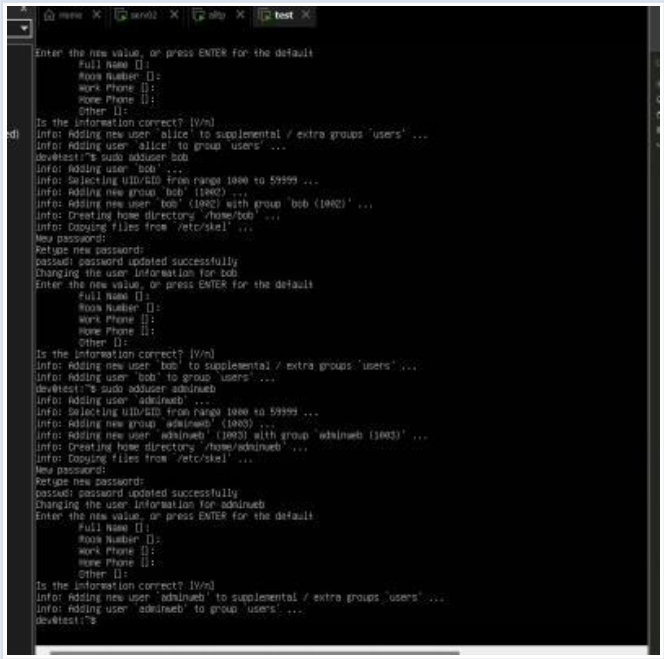
03 — COMPTES

Captures de création : première série

Les deux images présentent l'assistant adduser et la préparation des comptes. Refaire la vérification de chaque utilisateur sur chaque serveur.



Capture 04 — Création — série A.
TP original, p. 2. Assistant interactif et dossiers personnels.



Capture 05 — Création — série B.
TP original, p. 2. Autre séquence des comptes demandés.

Contrôles à rejouer

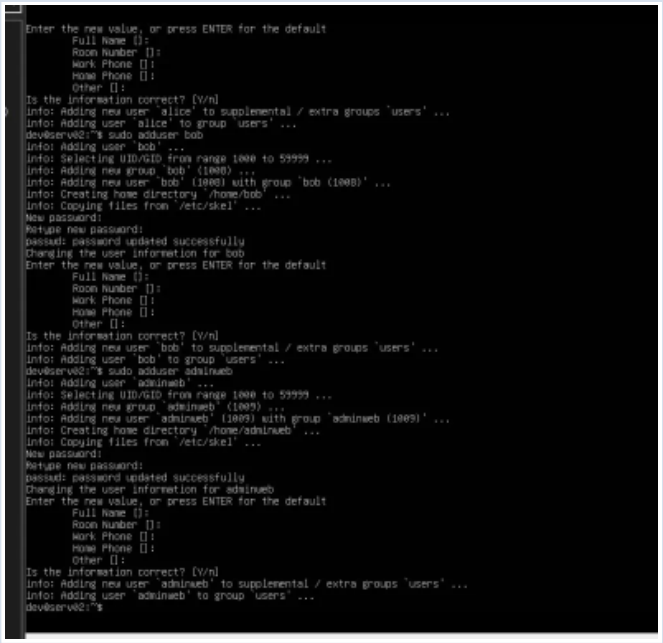
```
getent passwd alice bob adminweb
id alice
id bob
id adminweb
ls -ld /home/alice /home/bob /home/adminweb
```

Résultat attendu : chaque utilisateur possède un UID, un groupe principal et un répertoire personnel. Les UID des captures ne sont pas imposés et peuvent varier entre VM.

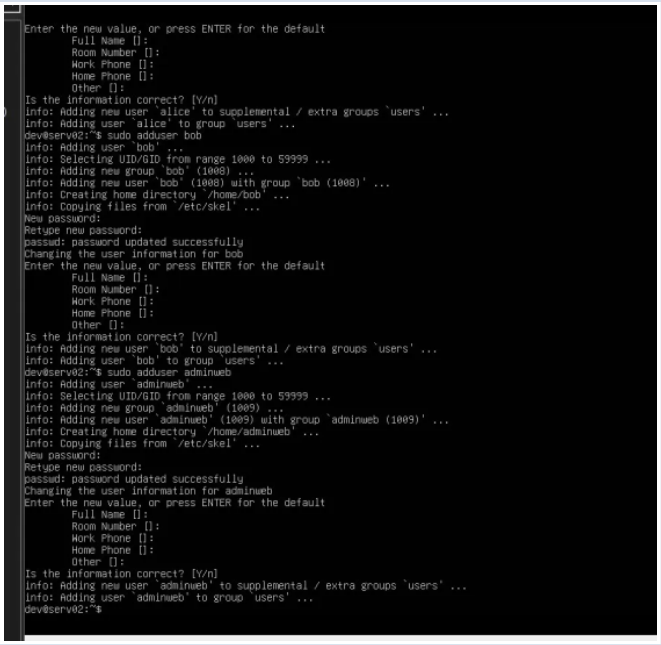
03 — COMPTES

Captures de création : seconde série

Deux autres captures liées aux comptes sont conservées, même si leur emplacement dans le TP est différent.



Capture 06 — Création — série C.
TP original, p. 2. Commandes adduser dans une autre session.



Capture 08 — Comptes en section Groupes.
TP original, p. 3. La création ne suffit pas à prouver les appartenances à web ou admins.

Vérifier une session utilisateur

```
su - alice
whoami
pwd
exit
```

Résultat attendu : whoami affiche alice et pwd correspond à son dossier personnel après une connexion de type login. Refaire ce contrôle pour les comptes utilisés au test collaboratif.

> **Compte et groupe sont distincts**
> La création de l'utilisateur, de son secret et de son dossier /home ne crée pas automatiquement le groupe web ni les droits d'administration.

03 — GROUPES

Fichiers de comptes et groupes du projet

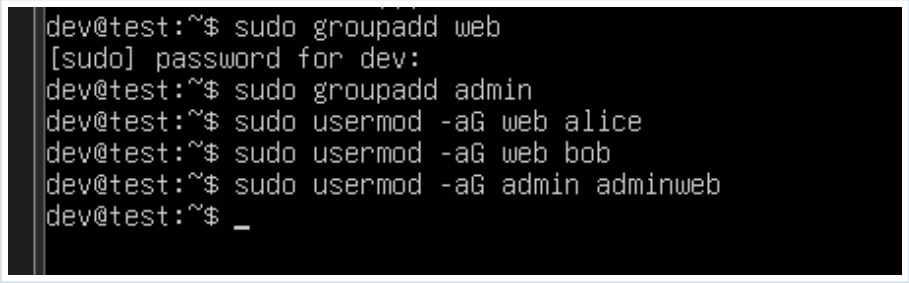


Capture 07 — Entrées locales des comptes. TP original, p. 3. Les comptes dev, alice, bob et adminweb montrent UID, GID, home et shell.

/etc/passwd décrit les comptes locaux, pas les mots de passe en clair. Leurs empreintes sont normalement dans **/etc/shadow**, protégé. **getent passwd** interroge toutes les sources d'identité configurées.

Création et affectation des groupes

```
sudo groupadd web
sudo groupadd admins
sudo usermod -aG web alice
sudo usermod -aG web bob
sudo usermod -aG admins adminweb
```

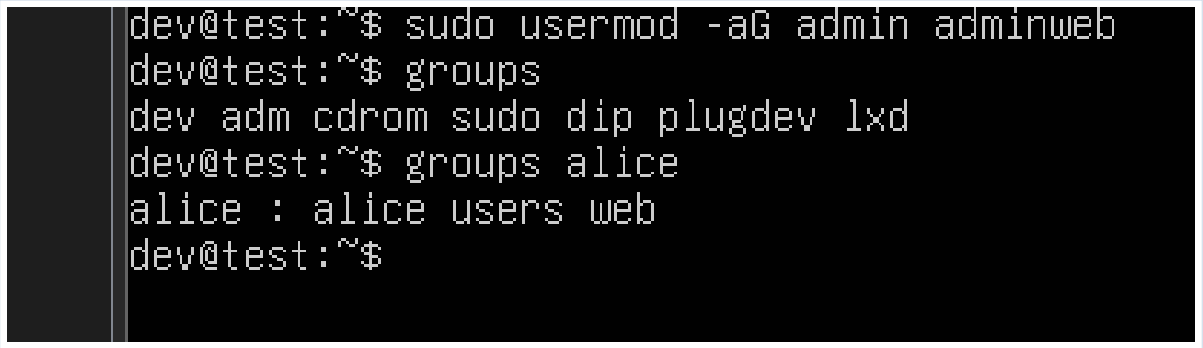


Capture 09 — Commandes de groupe historiques. TP original, p. 3. L'ancienne séquence utilise admin au singulier ; la consigne demande admins.

> **Point à corriger : admin / admins**
> Conserver **admins** dans la reprise. Si admin existe déjà, examiner ses usages avant modification. Sur un lab neuf, créer directement admins. **-aG** ajoute les groupes supplémentaires sans retirer les autres.

03 — PRIVILÈGES

Vérifier groupes et accès sudo



Capture 10 — Vérification groups alice. TP original, p. 4. web apparaît pour Alice. groups sans argument décrit le compte courant dev.

```
getent group web admins
groups alice
groups bob
groups adminweb
id adminweb
```

Le nom admins ne donne pas root

L'appartenance à **admins** ne confère pas automatiquement des droits root. Ils sont définis par sudoers ou un mécanisme équivalent. Si adminweb doit réellement administrer le lab, le compte autorisé peut l'ajouter au groupe sudo d'Ubuntu.

```
# Seulement si adminweb doit administrer les VM :
sudo usermod -aG sudo adminweb
# Dans sa nouvelle session :
sudo -l
```

> **Moindre privilège**
> Alice et Bob restent sans accès sudo. L'ajout au groupe sudo accorde des privilèges importants et doit être réservé à l'administration autorisée.

Vérification : Alice/Bob dans web ; adminweb dans admins. Fermer puis rouvrir les sessions pour renouveler les appartenances. Un résultat sur lb ne valide pas les deux autres machines.

04 — PERMISSIONS

Création de /srv/web et droit 2775

Objectif : créer le dossier de travail root:web avec l'écriture de groupe. Les dossiers créés sur différentes VM restent locaux.

```
sudo mkdir -p /srv/web
sudo chown root:web /srv/web
sudo chmod 2775 /srv/web
ls -ld /srv/web
stat -c "%A %a %U:%G %n" /srv/web
```

Chiffre	Signification	Effet
2	Bit setgid	Les nouveaux objets héritent du groupe web.
7	Propriétaire : rwx	root peut lire, créer et traverser.
7	Groupe : rwx	web peut lire, créer et traverser.
5	Autres : r-x	Lecture/liste et traversée, sans écriture.

> **Setgid n'est pas le sticky bit**
> Le 2 de **2775** représente **setgid**. Le sticky bit vaut 1 dans le chiffre spécial et limite notamment la suppression d'entrées sur un dossier partagé. Ces mécanismes sont différents.

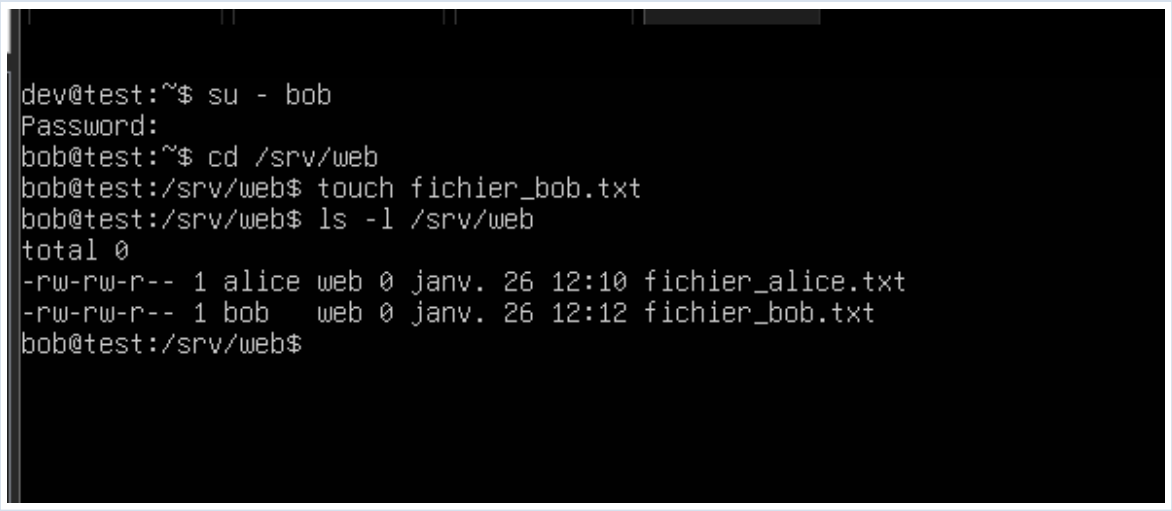
Résultat attendu : drwxrwsr-x root web, avec s à la position d'exécution du groupe. Le mode conserve une lecture aux autres ; un contenu privé demanderait une autre politique.

04 — COLLABORATION

Création et partage des fichiers

Objectif : vérifier l'héritage de groupe et l'écriture collaborative. Setgid ne garantit pas à lui seul que tous les fichiers sont modifiables par le groupe : le mode dépend aussi de l'umask.

```
# Nouvelle session de Alice :
su - alice
cd /srv/web
umask 0002
touch fichier_alice.txt
exit
# Nouvelle session de Bob :
su - bob
cd /srv/web
umask 0002
touch fichier_bob.txt
ls -l /srv/web
exit
```



Capture 11 — Fichiers de Alice et Bob. TP original, p. 4. Les créateurs diffèrent ; le groupe web et le mode rw-rw-r-- sont visibles.

Le groupe web est hérité grâce au setgid. Avec un mode de création usuel 0666 et umask 0002, le résultat est 0664. Umask 0022 donnerait généralement 0644, empêchant Bob de modifier le fichier d'Alice.

> **Portée du réglage**
> umask règle ici la session courante. Une politique permanente demande une configuration des sessions ou une ACL par défaut, puis un nouveau test.

04 — PUBLICATION

Dossier de travail et dossier publié

Objectif : distinguer **/srv/web**, dossier collaboratif, du DocumentRoot **/var/www/html**. Le TP ne relie pas automatiquement les deux.

Emplacement	Rôle
/home/alice et /home/bob	Fichiers et préférences personnels.
/srv/web	Travail partagé du groupe web.
/var/www/html	Ressources du site Apache par défaut.
/var/backups/projet3	Sauvegardes hors accès HTTP.

Publication proposée

1. Préparer et relire les fichiers de travail. Le lab publie un HTML et un CSS simples.

2. Sauvegarder l'ancien index hors du dossier web, sans écraser une sauvegarde existante.

3. Publier avec le compte d'administration. Apache doit pouvoir lire sans pouvoir modifier tout le site.

```
sudo install -d -m 700 /var/backups/projet3
sudo cp -n /var/www/html/index.html \
/var/backups/projet3/index.html.avant-tp
# Si une page a ete preparee dans /srv/web :
sudo install -o root -g root -m 644 \
/srv/web/index.html /var/www/html/index.html
```

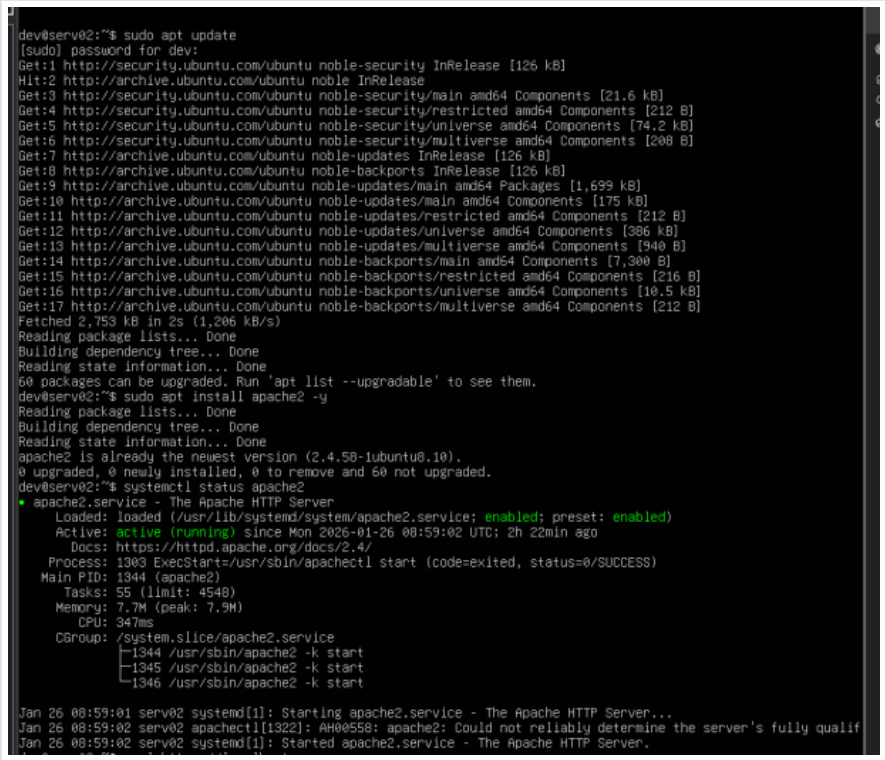
> **Pas de stockage partagé**
> Deux dossiers /srv/web sur deux VM ne sont pas synchronisés. La gestion de versions et la synchronisation sont des améliorations possibles, pas le résultat de chmod.

05 — APACHE2

Installation et première vérification

Objectif : installer Apache2 sur **web1 et web2**. Sur lb, HAProxy doit pouvoir écouter : ne pas laisser deux services se disputer la même IP et le port 80.

```
sudo apt update
sudo apt install apache2 curl -y
systemctl status apache2 --no-pager
sudo ss -lntp | grep ":80"
```



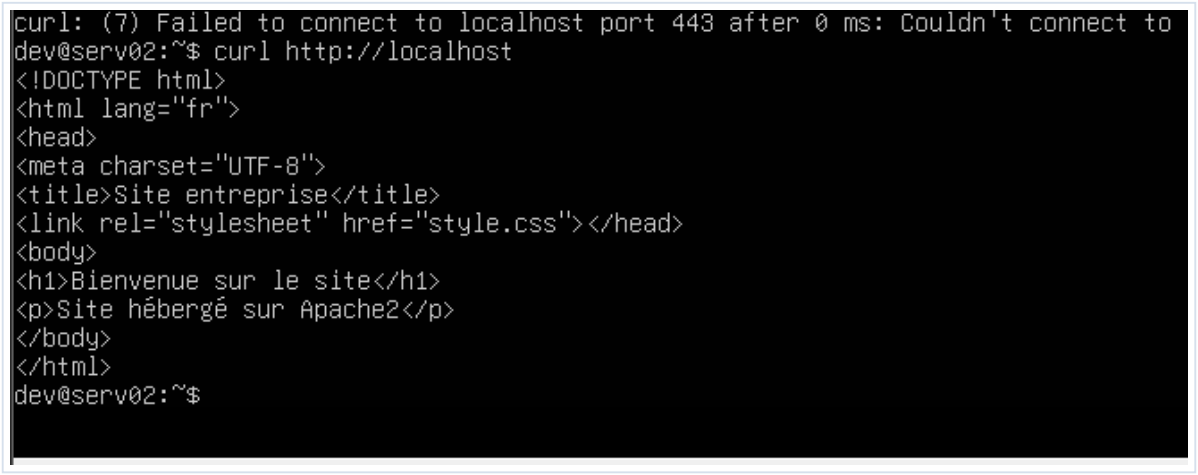
Capture 12 — Installation et état Apache. TP original, p. 5. Paquets installés et service actif sur une machine web.

Résultat attendu : active (running), TCP 80 à l'écoute et première réponse HTTP locale. apt update actualise les listes : il n'installe pas à lui seul toutes les mises à jour.

> **Accès aux dépôts**
> Faire les installations avec un accès NAT/DNS fonctionnel avant isolation. Ne pas changer arbitrairement les IP du projet pour contourner une panne de résolution.

05 — PROTOCOLES

HTTP local et port 443



Capture 13 — Curl local : HTTP et HTTPS. TP original, p. 5. Une tentative sur 443 échoue ; curl http://localhost retourne du HTML.

Test sur chaque backend

```
curl -I http://127.0.0.1/
curl http://127.0.0.1/
sudo apache2ctl configtest
sudo apache2ctl -S
```

Le service par défaut répond en HTTP/80. Un échec HTTPS/443 ne signifie pas nécessairement une panne d'Apache : aucun certificat ni site TLS n'est prévu ici.

DocumentRoot et journaux

```
ls -l /var/www/html
sudo grep -n DocumentRoot \
/etc/apache2/sites-available/000-default.conf
sudo journalctl -u apache2 -n 30 --no-pager
```

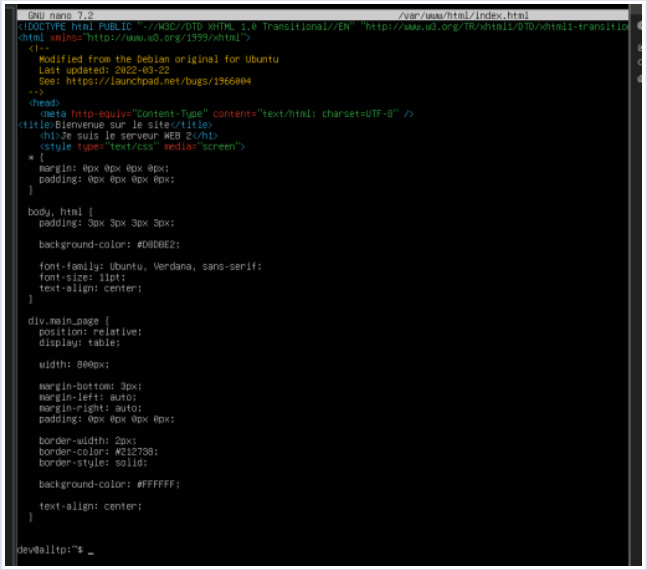
Valider la syntaxe avant recharge d'une configuration. Les modifications HTML/CSS ne nécessitent normalement pas de redémarrage du service.

> **Avertissement de nom complet**
> AH00558 peut apparaître alors qu'Apache répond. Un ServerName global adapté peut traiter cet avertissement ; il ne configure pas de DNS.

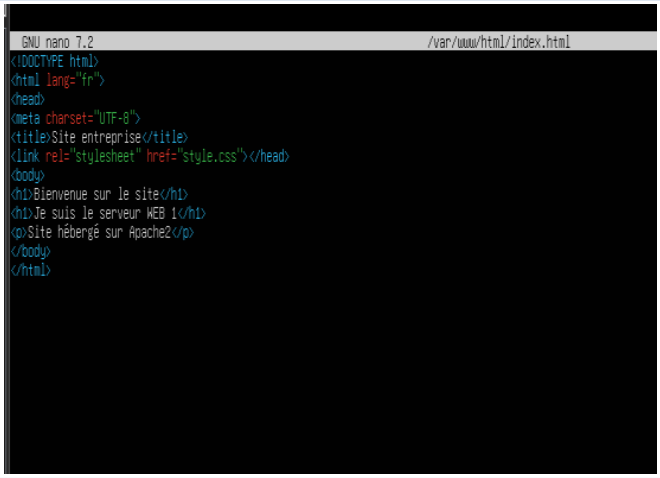
05 — HTML

Pages historiques et version harmonisée

Objectif : reconnaître le backend par son contenu. L'une des pages historiques conserve la page Ubuntu par défaut ; l'autre est une page simple.



Capture 14 — Page Ubuntu modifiée.
TP original, p. 5. Ajout d'un repère dans la page par défaut.



Capture 15 — Page HTML simple.
TP original, p. 5. Le code contient le repère WEB 1.

HTML proposé pour web1 (.142)

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Laboratoire Web - WEB 1</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Bienvenue sur le site</h1>
  <h2>Je suis le serveur WEB 1</h2>
  <p>Site heberge sur Apache2</p>
</body>
</html>
```

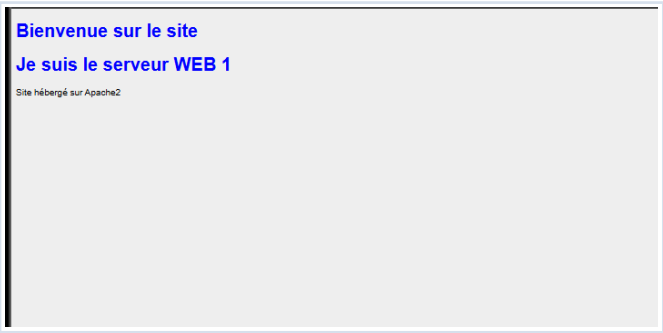
Enregistrer dans **/var/www/html/index.html**. Sur **web2 (.136)**, remplacer les deux identifiants par WEB 2. Cette proposition n'est pas une retouche des images.

05 — CSS

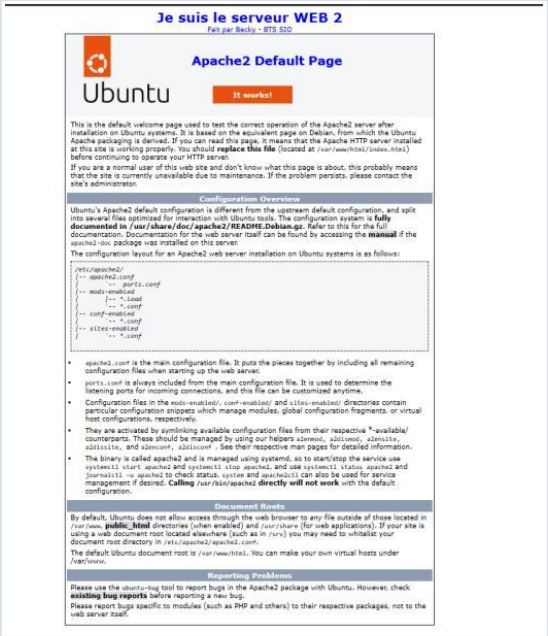
Feuille de style et affichage

Créer **/var/www/html/style.css** sur les deux backends pour harmoniser la présentation, en conservant leur identifiant propre.

```
body {
  background-color: #eeeeee;
  font-family: Arial, sans-serif;
}
h1, h2 {
  color: blue;
}
```



Capture 16 — Page simple : WEB 1.
TP original, p. 6. Repère de serveur visible.



Capture 19 — Page Ubuntu : WEB 2.
TP original, p. 7. Repère ajouté au-dessus de la page par défaut.

Permissions et contrôle CSS

```
ls -l /var/www/html/index.html /var/www/html/style.css
sudo chmod 644 /var/www/html/index.html /var/www/html/style.css
curl -I http://127.0.0.1/style.css
```

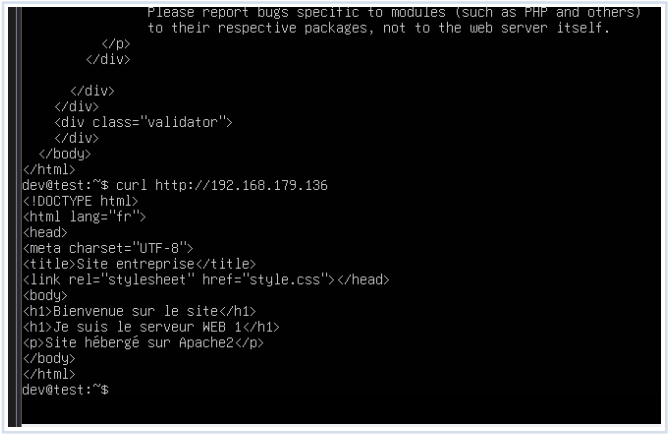
Résultat attendu : fichiers lisibles par Apache ; HTTP 200 pour le CSS. Actualiser sans cache dans le navigateur. Ne pas appliquer chmod 777 pour traiter une erreur de lecture.

05 — TESTS DIRECTS

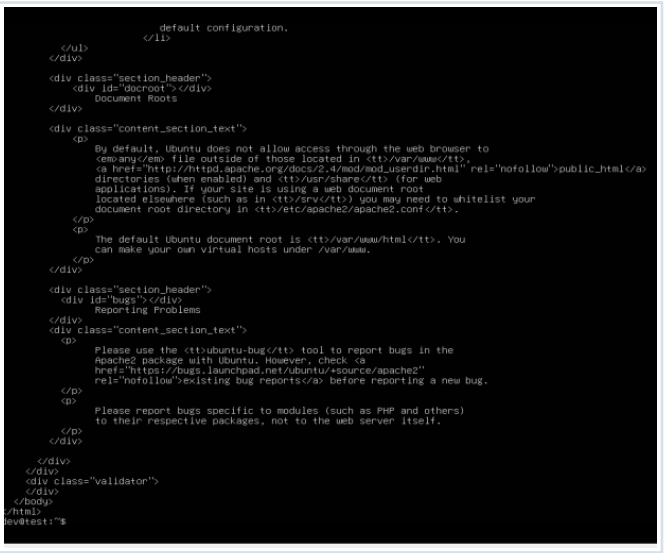
Vérification HTTP des deux backends

Objectif : contrôler Apache indépendamment de HAProxy, depuis lb ou un client autorisé. Utiliser les IP du plan .179.

```
curl -I http://192.168.179.142/
curl -I http://192.168.179.136/
curl -sS http://192.168.179.142/ | grep "serveur WEB"
curl -sS http://192.168.179.136/ | grep "serveur WEB"
```



Capture 17 — HTML direct de .136.
TP original, p. 6. L'ancien titre WEB 1 est incohérent avec web2 = .136.



Capture 18 — HTML de la page Ubuntu.
TP original, p. 7. Un contenu retourné ne prouve pas la traversée du proxy.

> **Résultat attendu dans la reprise**
> .142 doit retourner WEB 1 et .136 WEB 2. Les différences historiques restent dans les captures, mais ne doivent pas être recopiées dans la reprise.

Les adresses 192.168.1.10 et .11 dans les consignes sont des exemples d'un autre plan : elles ne remplacent pas les IP réelles retenues.

06 — HAPROXY

Installation sur la machine lb

Objectif : installer l'équilibrage sur **lb (.143)**, sans remplacer Apache sur web1 et web2.

```
sudo apt update
sudo apt install haproxy curl -y
haproxy -v
sudo ss -lntp | grep ":80"
```

1. Vérifier l'IP .143 et les réponses HTTP de .142 et .136.

2. Installer HAProxy et noter la version réellement obtenue.

3. Contrôler qu'aucun autre service ne monopolise .143:80. Si Apache a été installé sur lb par erreur, examiner son usage avant arrêt ou retrait.

4. Sauvegarder la configuration et conserver les sections global et defaults utiles.

```
sudo cp -n /etc/haproxy/haproxy.cfg \
/etc/haproxy/haproxy.cfg.avant-projet3
sudo nano /etc/haproxy/haproxy.cfg
```

Rôle du proxy

Le client contacte lb ; HAProxy ouvre une connexion vers le backend choisi. Le client n'a pas à choisir lui-même web1 ou web2 pour chaque requête.

> **Aucune redondance du proxy**
> La panne d'un backend peut être tolérée ; la panne de lb reste bloquante. Un second proxy et une IP virtuelle seraient un autre déploiement.

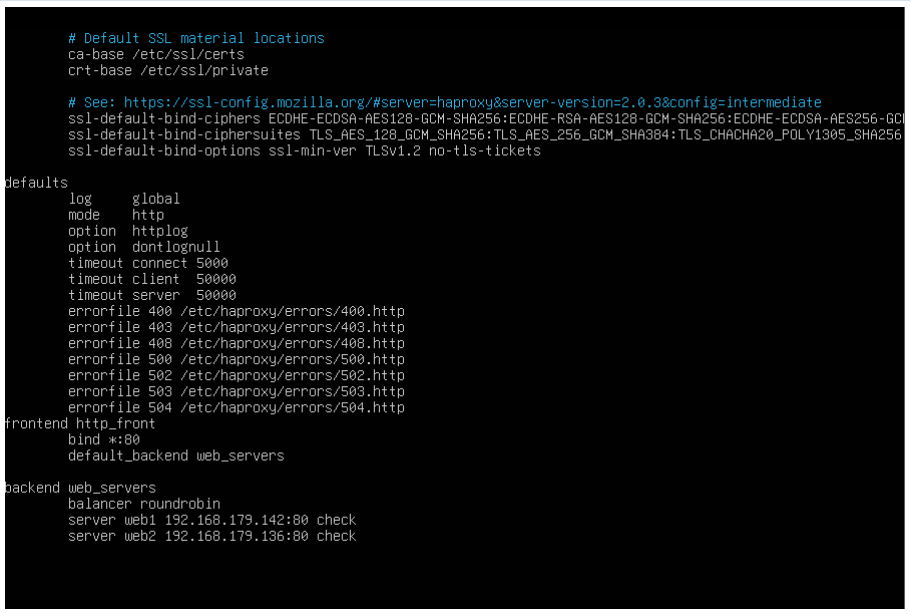
06 — CONFIGURATION

Frontend HTTP et backend de deux serveurs

Objectif : écouter .143:80 et sélectionner un Apache disponible. La reprise conserve les IP visibles dans la configuration et ajoute un contrôle HTTP explicite.

```
frontend http_front
    bind 192.168.179.143:80
    mode http
    default_backend web_servers

backend web_servers
    mode http
    balance roundrobin
    option httpchk GET /
    http-check expect status 200
    server web1 192.168.179.142:80 check inter 2s fall 3 rise 2
    server web2 192.168.179.136:80 check inter 2s fall 3 rise 2
```



Capture 20 — Configuration HAProxy historique. TP original, p. 8. Les backends sont .142 et .136 ; bind *:80 écoute historiquement toutes les adresses.

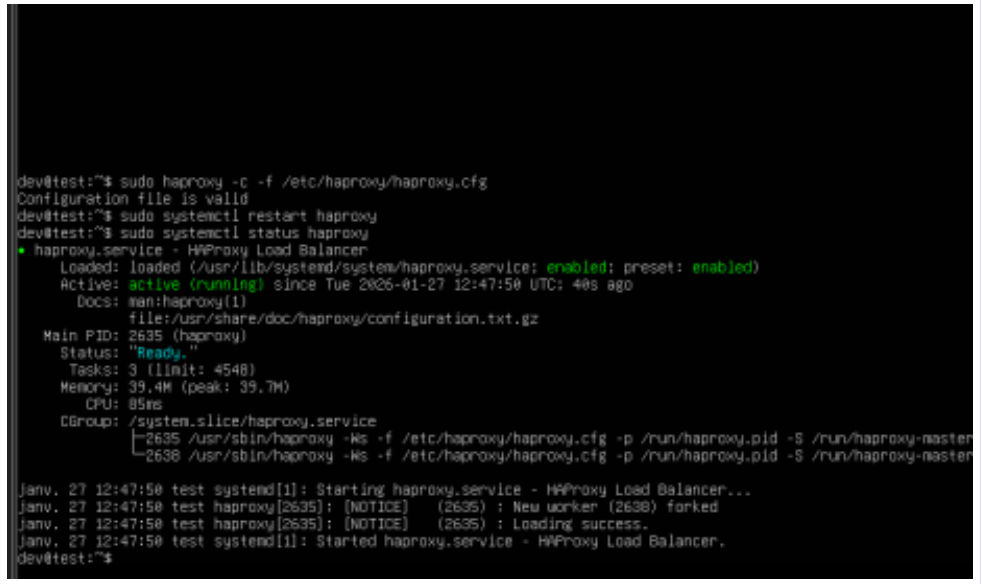
> **Conditions de reprise**
> Ne pas créer d'écoute concurrente ni de seconde section defaults contradictoire. Vérifier les timeouts connect, client et server. Le health check attend 200 : l'adapter si le site redirige volontairement.

06 — MISE EN SERVICE

Validation et démarrage du proxy

Objectif : valider la syntaxe puis vérifier l'écoute et les réponses. Un service actif peut encore n'avoir aucun backend utilisable.

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
# Continuer seulement si la validation reussit :
sudo systemctl enable --now haproxy
# Pour appliquer une modification sur le service actif :
sudo systemctl reload haproxy
systemctl status haproxy --no-pager
sudo ss -lntp | grep ":80"
```



Capture 21 — Statut HAProxy sur test. TP original, p. 8. Le service est active (running) sur la machine lb.

Interprétation

- La validation ne teste pas la disponibilité complète des backends.
- Le statut actif prouve le processus, pas l'accès au site.
- La réponse via .143 et les journaux apportent les preuves fonctionnelles.

```
curl -I http://192.168.179.143/
sudo journalctl -u haproxy -n 40 --no-pager
```

enable --now démarre le service au premier usage. Sur un service déjà actif, une recharge après validation applique les changements compatibles ; un redémarrage est une alternative si nécessaire.

07 — RÉPARTITION

Tester lb, pas les adresses des backends

Objectif : démontrer la répartition avec **une même URL .143**. Les images finales sont associées à .142 et .136 dans le TP : elles ne suffisent pas à établir le passage par lb.



Capture 22 — Affichage associé à .142.
TP original, p. 8. WEB 2 est incohérent avec le rôle web1.



Capture 23 — Affichage associé à .136.
TP original, p. 9. WEB 1 est incohérent avec le rôle web2.

Série de requêtes depuis le client

```
curl -sS http://192.168.179.143/ | grep "serveur WEB"
for i in $(seq 1 10); do
  curl -sS -H "Connection: close" http://192.168.179.143/ \
    | grep "serveur WEB"
done
```

> **Preuve à produire**
> Un terminal doit montrer les deux identifiants retournés par **.143**. Conserver une capture navigateur avec la barre d'adresse. Rafraîchir .142 puis .136 ne prouve pas l'équilibrage.

Round-robin répartit les requêtes éligibles entre les serveurs disponibles. Des requêtes comparables peuvent donner un partage proche de 50/50 ; cache, connexions et ressources annexes empêchent d'en faire une garantie générale.

07 — TEST DE PANNE

Arrêt contrôlé d’un serveur web

Objectif : vérifier les nouvelles réponses via lb quand Apache de web1 est arrêté. Ce test est proposé pour la reprise : aucune mesure complète n’est fournie par le TP.

1. Confirmer que les deux backends répondent et que .143 retourne les deux identifiants.

2. Sur **web1**, arrêter Apache dans le lab. Ne pas exécuter cette commande sur lb ou en production.

3. Attendre la détection, puis répéter les requêtes vers **.143** : WEB 2 doit être retourné.

4. Restaurer Apache sur web1. Vérifier sa réponse directe et son retour dans les cibles de HAProxy.

```
# Sur web1 :
sudo systemctl stop apache2
# Depuis le client, apres detection :
curl -sS http://192.168.179.143/ | grep "serveur WEB"
# Puis sur web1 :
sudo systemctl start apache2
curl -I http://127.0.0.1/
```

Délais et erreurs possibles

Avec inter 2s, fall 3 et rise 2, plusieurs échecs ou succès sont nécessaires. La détection est de l’ordre de plusieurs secondes, pas instantanée ; les timeouts et le moment de la panne influencent le délai.

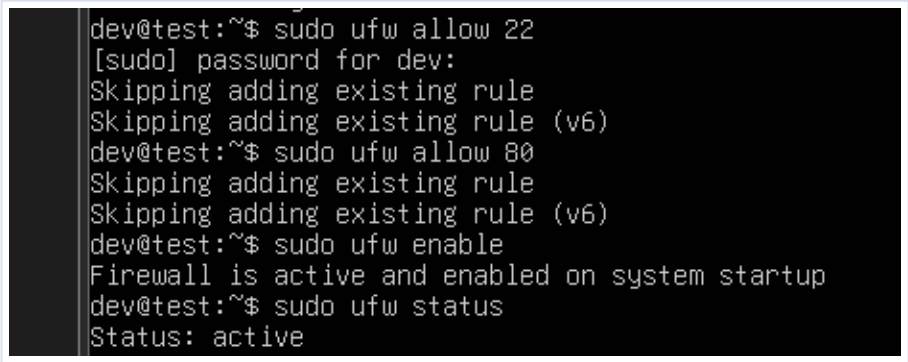
> **Pas de promesse zéro interruption**
> Une requête en cours peut échouer. Si les deux backends sont indisponibles, le proxy renvoie typiquement 503. Si lb tombe, l’accès par lb est impossible.

Preuves : arrêt de web1, réponses WEB 2 via .143, journaux et retour WEB 1. Consigner les heures réelles et les erreurs, sans inventer de durée mesurée.

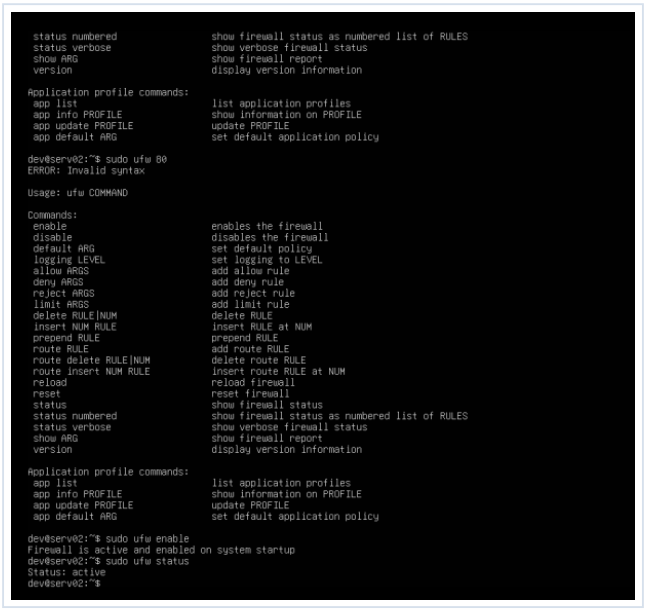
08 — PARE-FEU

UFW : activation et précautions

Objectif : filtrer les accès tout en gardant les flux nécessaires. L'énoncé ouvre 22 et 80 ; la reprise précise les sources autorisées.



Capture 24 — UFW sur test. TP original, p. 9. Les règles 22 et 80 existent déjà ; le pare-feu est actif.



Capture 25 — UFW : aide et état. TP original, p. 9. Une invocation invalide affiche l'aide ; le statut final active ne détaille pas toutes les règles.

> **Avant activation**
> Utiliser la console et préparer l'accès SSH. Un pare-feu actif ne prouve pas les bonnes règles ni la disponibilité HTTP.

```
sudo ufw status numbered
sudo ufw status verbose
```

08 — FILTRAGE

Règles ciblées pour le laboratoire

Les règles suivantes sont des **exemples de reprise**. Examiner les règles déjà présentes : une règle restrictive ajoutée ne retire pas une ancienne ouverture générale.

Sur lb : HTTP du réseau client privé

```
sudo ufw allow from 192.168.179.0/24 to any port 80 proto tcp
# Si le reseau prive est bien celui d'administration :
sudo ufw allow from 192.168.179.0/24 to any port 22 proto tcp
sudo ufw enable
sudo ufw status verbose
```

Sur web1 et web2 : HTTP depuis lb

```
sudo ufw allow from 192.168.179.143 to any port 80 proto tcp
# Adapter SSH au vrai poste d'administration :
sudo ufw allow from 192.168.179.0/24 to any port 22 proto tcp
sudo ufw enable
sudo ufw status verbose
```

Les tests directs des backends peuvent être faits depuis lb, autorisé. Si un client distinct doit les interroger, ajouter une exception ciblée et documentée, puis la retirer si elle n'est plus utile.

Autres précautions

- Ne pas publier les services sur Internet pour l'atelier.
- Utiliser des mots de passe robustes et éviter les privilèges excessifs.
- Conserver les sauvegardes hors du site publié.
- Ne pas rendre le site modifiable par tous ou par Apache sans besoin.
- Si des statistiques sont ajoutées plus tard, limiter leur accès aux administrateurs.

> **Hors périmètre**
> HTTPS, redondance du proxy, stockage partagé et supervision centralisée ne sont pas déployés ici.

09 — RECETTE

Tests et preuves attendues

Ce plan doit être rejoué sur les VM recréées ; les résultats attendus ne sont pas des mesures effectuées pendant la génération.

Test	Résultat attendu	Preuve
Réseau	lb .143 ; web1 .142 ; web2 .136 ; /24	ip -br address et pings
Comptes	Trois comptes sur chaque VM	getent passwd et id
Groupes	Alice/Bob dans web ; adminweb dans admins	groups sur les trois VM
Administration	sudo uniquement si autorisé	sudo -l dans sa session
Dossier	root:web ; mode 2775 ; setgid	stat et ls -ld
Fichiers	web ; écriture avec umask 0002	ls -l et test collaboratif
Apache	Actif ; HTTP 200 par backend	status et curl direct
Repères	WEB 1/.142 ; WEB 2/.136	HTML ou navigateur
HAProxy	Syntaxe valide	haproxy -c
Frontend	Écoute .143:80 et HTTP	ss et curl
Répartition	Deux identifiants via .143	Série de requêtes
Panne web1	WEB 2 ; retour WEB 1	Journaux et sorties
Pare-feu	Accès utiles conservés	UFW et test HTTP

Test collaboratif complémentaire
La création de fichiers vides ne prouve pas l'écriture de Bob dans le fichier d'Alice. Tester dans une session de Bob après contrôle des groupes.

```
printf "Ajout de Bob\n" >> /srv/web/fichier_alice.txt
cat /srv/web/fichier_alice.txt
```

09 — DIAGNOSTIC

Corrections et dépannage

Écart / symptôme	Interprétation	Action
192.168.1.10 / .11	Autre plan que le réseau du lab	Conserver .142 et .136
WEB 1 associé à .136	Titres inversés	Harmoniser noms et IP
Test sur .142 / .136	Accès direct, pas preuve de proxy	Tester la même URL .143
admin / admins	Nom de groupe différent	Suivre admins dans le lab neuf
2775 nommé sticky	Le 2 est setgid	Expliquer héritage et umask
Bob ne peut pas écrire	Mode / umask inadaptés	Contrôler id et ls -l
HTTPS local échoue	TLS non configuré	Tester HTTP/80
HAProxy ne démarre pas	Syntaxe, IP ou port occupé	haproxy -c, ip, ss
HAProxy actif, HTTP 503	Aucun backend utilisable	Apache, UFW, health checks
UFW affiche l’aide	Commande incorrecte	Séparer les commandes

Ordre de recherche

1. Réseau/IP ; **2.** HTTP direct ; **3.** syntaxe du proxy ; **4.** écoute .143:80 ; **5.** UFW ; **6.** HTTP via lb ; **7.** journaux et santé. Ne pas désactiver toutes les protections pour masquer une panne.

Limites des captures
Le TP ne démontre pas une série de requêtes sur .143 ni un arrêt de backend mesuré. Le livrable conserve les preuves historiques et ajoute les étapes pour produire ces validations.

10 — QUESTIONS

Réponses aux questions du TP

1. Où sont stockés les utilisateurs sous Linux ?

Les comptes locaux sont décrits dans /etc/passwd : nom, UID, GID, informations, home et shell. Les empreintes des mots de passe locaux sont généralement dans /etc/shadow, protégé. getent passwd peut aussi consulter les sources d'identité configurées.

2. Quel est le rôle de /home ?

/home contient habituellement les dossiers personnels des utilisateurs ordinaires, leurs fichiers et préférences. root utilise généralement /root ; les comptes techniques peuvent utiliser d'autres emplacements.

3. Pourquoi utiliser des groupes ?

Les groupes attribuent des permissions à plusieurs personnes sans gérer les droits individuellement sur chaque ressource. web donne à Alice et Bob leurs accès à /srv/web. Il faut renouveler les sessions après changement des appartenances.

4. À quoi sert le load balancing ?

Il répartit les requêtes entre les serveurs disponibles et peut tolérer la panne d'un backend. Il ne supprime pas le point de panne du proxy unique ni la nécessité de synchroniser les données d'une application.

5. Quelle différence entre Apache et HAProxy ?

Apache sert les fichiers HTML/CSS ; HAProxy reçoit les requêtes clients et les transmet au backend sélectionné. Dans cette architecture, l'un publie et l'autre répartit. Leurs fonctionnalités sont plus larges dans d'autres configurations.

6. Que se passe-t-il si web1 est arrêté ?

Après détection, HAProxy retire web1 et transmet les nouvelles requêtes à web2 si disponible. Une requête en cours peut échouer. Si lb ou les deux backends tombent, cette architecture ne peut plus fournir le site via lb.

10 — BILAN

Conclusion et préparation du rendu

Ce projet rassemble les bases de l'administration Linux et de l'équilibrage HTTP : comptes, groupes, droits Unix, setgid, Apache2 et frontend HAProxy avec deux backends.

Les captures montrent des opérations sur les identités et le réseau, des fichiers de groupe dans /srv/web, des contenus Apache distincts, une configuration HAProxy et des services actifs. Elles ont aussi permis de relever des incohérences d'identifiants, de tests et d'explication des droits.

> **Bilan de validation**
> Le livrable intègre **les 25 images du TP**. Les preuves de répartition via .143 et de panne restent à produire sur les VM. Aucune redondance Heartbeat ni continuité sans interruption n'est affirmée.

Améliorations possibles

- Second proxy avec reprise d'IP et mécanisme évitant le split-brain.
- HTTPS et certificats adaptés ; gestion des secrets.
- Health checks applicatifs, supervision et alertes.
- Gestion de versions et synchronisation maîtrisée des contenus.
- Sauvegardes testées et revue des droits d'administration.

Avant remise

- Compléter établissement, année et date sur la couverture.
- Indiquer l'Ubuntu, les versions des services et les ressources retenues.
- Rejouer la recette page 28 et consigner les résultats.
- Harmoniser WEB 1/.142 et WEB 2/.136 dans les nouvelles images.
- Ajouter une preuve de requêtes via .143 et du retrait/retour d'un backend.

Compétences : virtualisation, IPv4, identités locales, groupes, permissions, systemd, HTTP, proxy, santé des services, UFW et diagnostic.

ANNEXE — VISUELS

Index des 25 captures originales

Les 25 images du PDF de TP sont toutes intégrées une fois, sans retouche de leur contenu. Les corrections figurent dans le texte.

N°	Capture	TP	Livrable
01	IP et pings depuis alltp	p. 1	p. 6
02	IP et pings depuis test	p. 1	p. 6
03	IP et pings depuis serv02	p. 1	p. 7
04	Création des comptes — série A	p. 2	p. 9
05	Création des comptes — série B	p. 2	p. 9
06	Création des comptes — série C	p. 2	p. 10
07	Entrées locales des comptes	p. 3	p. 11
08	Comptes en section Groupes	p. 3	p. 10
09	Groupes web et admin	p. 3	p. 11
10	Vérification groups alice	p. 4	p. 12
11	Fichiers dans /srv/web	p. 4	p. 14
12	Installation et statut Apache	p. 5	p. 16
13	Curl HTTP et échec port 443	p. 5	p. 17
14	Page Ubuntu modifiée	p. 5	p. 18
15	HTML simple dans Nano	p. 5	p. 18
16	Affichage de WEB 1	p. 6	p. 19
17	HTML retourné par .136	p. 6	p. 20
18	HTML de la page Ubuntu	p. 7	p. 20
19	Affichage de WEB 2	p. 7	p. 19
20	Configuration HAProxy	p. 8	p. 22
21	Statut HAProxy sur test	p. 8	p. 23
22	Affichage associé à .142	p. 8	p. 24
23	Affichage associé à .136	p. 9	p. 24
24	UFW actif sur test	p. 9	p. 26
25	UFW : aide et état	p. 9	p. 26

Figure A, page 4 : schéma original créé pour expliquer le projet. Les commandes ont été imprimées comme procédure, pas exécutées sur les VM de Rebecca par l'assistant.